Command Palette (Cmd+K)

Problem: Build a command palette: a keyboard-first modal that opens with Cmd+K / Ctrl+K, fuzzy-searches a list of commands, lets the user move with arrow keys and run a command with Enter.

Often asked at: Atlassian, Notion, Vercel

Tech stack: React (Vite, JavaScript) (default, none given)

Time limit: 60 minutes (default, none given)

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements:** functional, non-functional, clarifying questions, assumptions.
2. **List interactions / use cases:** every input and what the user should see. This becomes the checklist.
3. **Design the architecture:** a few components with one job each.
4. **Design the state:** the smallest state; derive everything else.
5. **Design data and contracts:** shape of the data, props, function signatures.
6. **Trace one data flow end to end** before coding.
7. **Find the hard parts** and decide how to solve them up front.
8. **Build in layers:** a working version first, then one feature at a time, each one runnable.
9. **Cover states and edge cases:** empty, error, boundaries.
10. **Wrap up:** trade-offs, what you would add with more time.

Time budget for 60 minutes

Time	Activity
0-5 min	Requirements, clarifying questions, assumptions
5-10 min	Use cases, components, state, data shape
10-20 min	Static palette: open/close, input, list of commands
20-35 min	Fuzzy search + highlight, keyboard navigation, run command
35-45 min	Focus management, a11y (combobox/listbox), scroll into view
45-55 min	Recent commands, empty state, polish, edge cases
55-60 min	Walk through trade-offs and extensions

Think out loud

Say every decision and its reason as you make it: *"I'll use the native `<dialog>` with `showModal()` because it gives me the focus trap, the backdrop and Escape for free."* The interviewer grades the reasoning as much as the code. If you are stuck, say what you are trying and what you will fall back to.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Open the palette with **Cmd+K (Mac) / Ctrl+K (Windows/Linux)** from anywhere on the page, and with a visible button.
- Pressing the shortcut again, **Escape**, or a **click on the backdrop** closes it.
- A search input is focused as soon as it opens.
- Show all commands grouped by section when the query is empty, with **recently used** commands on top.
- **Fuzzy search**: typing `gts` finds *Go to Settings*. Matched characters are highlighted. Best match first.
- Search also matches hidden **keywords** (typing `theme` finds *Toggle dark mode*).
- **Arrow Up / Down** move the active item (wrapping around), **Enter** runs it, **mouse hover + click** do the same.
- Running a command closes the palette and performs the action (navigate, toggle theme, show a toast...).
- Show an **empty state** when nothing matches.

Non-functional requirements

Area	What it means here
Performance	Filtering runs on every keystroke; must stay instant for a few hundred commands. Only the active row re-styles.
Scalability	Commands are data, so any feature can register more without touching the palette.
Error handling	`localStorage` may be blocked (private mode); clipboard can fail. Neither may crash the palette.
Accessibility	Focus trap while open, focus returned to the opener on close, `combobox` + `listbox` + `aria-activedescendant`, works with keyboard only, IME safe.
Reusability	`CommandPalette` takes `commands` and `onClose`; it knows nothing about the app.
Extensibility	Easy to add nested pages, async sources, per-command shortcuts later.

Clarifying questions

1. Is the list of commands static, or does it come from a server / change with the current page?
2. How smart must the search be: substring, fuzzy subsequence, or typo tolerant?
3. Do we need nested commands (e.g. *Change theme → Light / Dark*), or a flat list?
4. Should recently used commands be remembered across reloads?
5. Should the palette show per-command shortcuts and make them work globally?
6. Roughly how many commands: tens, hundreds, or thousands (e.g. searching all files)?

Assumptions we will build with

- Commands are **static, client-side**, about 10-500 items. No server.
- **Fuzzy subsequence** matching with simple scoring; no typo tolerance.
- **Flat list**, grouped by section. Nested pages are an extension.
- The last **5 recent commands** persist in `localStorage`.
- Only the palette shortcut itself (Cmd/Ctrl+K) is global.
- Modern browsers (native `<dialog>` is supported everywhere current).

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Press Cmd+K / Ctrl+K anywhere	Palette opens as a modal, input is focused, browser's own Ctrl+K is prevented
Press Cmd+K / Ctrl+K while open	Palette closes
Click "Search commands" button	Palette opens
Open with empty query	Recent group (if any) then all commands grouped by section; first row active
Type `gts`	Only matching commands, best first; `G`, `t`, `S` highlighted; first row active
Type `theme` (only in keywords)	*Toggle dark mode* shows, without highlight
Type `zzzq`	"No commands match “zzzq”", Enter does nothing
Arrow Down / Up	Active row moves; wraps from last to first and back; active row scrolls into view
Hover a row	That row becomes active
Enter / click on a row	Palette closes, command runs, command saved to recents
Escape / backdrop click	Palette closes, nothing runs
Tab while open	Focus stays in the input
Close by any route	Focus returns to the element that had it before opening

Action / input	Expected behavior / output
Reopen	Query is empty, active row is the first one, last run command is in "Recent"
Typing Japanese / Chinese with an IME and pressing Enter	Enter confirms the composition; no command runs
`localStorage` blocked	Palette works; recents simply are empty

---

3. Design the architecture

Module	Responsibility
`App`	Owns `open`, page, theme, toast. Registers Cmd+K. Builds the command list with app actions.
`useHotkey(key, handler)`	Global Cmd/Ctrl+key listener with cleanup.
`createCommands(actions)`	Returns the command data; each `perform` closes over injected actions.
`CommandPalette`	The modal: input, results, keyboard nav, focus management, runs commands.
`Highlight`	Renders a title with matched characters wrapped in `<mark>`.
`fuzzy.js`	Pure functions: `fuzzyMatch(query, text)`, `searchCommands(commands, query)`.
`recent.js`	`loadRecent()` / `saveRecent(id)` with safe `localStorage` access.

```
App
├── useHotkey('k', toggle)
├── createCommands({ navigate, toggleTheme, notify })   → commands[]
└── {open && <CommandPalette commands onClose />}
        ├── <dialog> (native modal: top layer, inert page, Esc)
        ├── <input role="combobox">  ── query ──► searchCommands() (fuzzy.js)
        ├── <ul role="listbox">
        │      └── <li role="option"> → <Highlight text indices />
        └── recent.js (load on mount, save on run)
```

Start simple: everything could live in one file first. The pure search logic is split out early because it is the part worth unit testing, and the hook because it is reusable.

---

4. Design the state / data model

State or field	Owner	Purpose
`open`	`App`	Is the palette mounted. The hotkey and the button both live in `App`.

State or field	Owner	Purpose
`page` , `theme` , `toast`	`App`	Demo app state the commands change.
`query`	`CommandPalette`	Current search text.
`activeIndex`	`CommandPalette`	Which row is highlighted (index into `items`).
`recentIds`	`CommandPalette`	Recent command ids, read once on mount.
`items`	derived (`useMemo`)	The rows to render: recents + grouped commands, or ranked results.

```
// App
const [open, setOpen] = useState(false);

// CommandPalette (mounted only while open)
const [query, setQuery] = useState('');
const [activeIndex, setActiveIndex] = useState(0);
const [recentIds] = useState(loadRecent); // lazy init: read storage once

const items = useMemo(
  () => (query.trim() ? searchCommands(commands, query) : recentPlusAll),
  [commands, query, recentIds]
);
```

Senior notes

- **Filtered results are derived, never stored.** Storing them would mean keeping two copies of the truth in sync on every keystroke.
- **The active item is an index, not a copy of the command.** It is reset to `0` in the `onChange` handler (not in an effect), so results and selection change in the same render.
- **Mount only while open** (`{open && <CommandPalette/>}`) so `query` and `activeIndex` reset for free on every open. No "reset" effect.
- `open` lives in `App` because the trigger (hotkey + button) lives there. Everything about searching lives inside the palette.

---

5. Design the data structures and contracts

Command shape

```
{
  id: 'nav-settings',            // stable id: React key, recents, aria ids
  title: 'Go to Settings',        // what is shown and highlighted
  section: 'Navigation',          // group header
  keywords: ['preferences', 'config'], // hidden search terms
```

```
  perform: () => navigate('Settings'), // what Enter / click does
}
```

Why stable ids: titles can change or be translated, but recents in `localStorage` and React keys must survive that.

Contracts

```
// Pure search
fuzzyMatch(query: string, text: string)
  → { score: number, indices: number[] } | null   // null = no match
fuzzyMatch('gts', 'Go to Settings') → { score: 23, indices: [0, 3, 6] }

searchCommands(commands: Command[], query: string)
  → Array<{ command, score, indices }>  // sorted best first

// Recents
loadRecent() → string[]          // [] if missing, corrupt or blocked
saveRecent(id: string) → string[] // most recent first, max 5, de-duplicated

// Component
<CommandPalette commands={Command[]} onClose={() => void} />

// Hook
useHotkey(key: string, handler: () => void) // handler must be stable
```

There is no API. If commands came from a server, the contract would be `GET /commands?q=...` returning the same `Command` shape without `perform`, and `perform` would be looked up by `id` in a client-side action map.

---

6. Trace the data flow

Key interaction: **user presses Ctrl+K, types `dark`, presses Enter.**

```
keydown Ctrl+K on window
   ↓  useHotkey: metaKey||ctrlKey && key==='k' → preventDefault()
setOpen(true) in App
   ↓  render
<CommandPalette> mounts → effect: remember opener, dialog.showModal(), input.focus()
   ↓
user types "dark" → onChange: setQuery('dark'), setActiveIndex(0)
   ↓  render
useMemo: searchCommands(commands, 'dark')
   → [{ command: Toggle dark mode, indices: [7,8,9,10], score }]
```

```
   ↓
<li aria-selected="true"> Toggle <mark>d</mark><mark>a</mark><mark>r</mark><mark>k</mark> mode
input aria-activedescendant → that li's id
   ↓
keydown Enter → run(items[0])
   ↓  saveRecent('pref-theme') → localStorage
onClose() → App setOpen(false)
command.perform() → toggleTheme() → setTheme('dark')
   ↓  render
<CommandPalette> unmounts → effect cleanup: dialog.close(), opener.focus()
<html data-theme="dark">, focus back on the trigger
```

---

7. Important technical challenges

Challenge 1: Fuzzy matching and ranking

Problem: A plain `includes` fails for `gts` → *Go to Settings*. But "any subsequence" alone ranks badly: `set` matches both *Go to **Set**tings* and *Show keyboard **s**hortcu**t**s*, and the first is clearly better.

Solution: Subsequence match, case-insensitive, scored with bonuses that match how people type:

- +1 per matched character
- +5 if it follows the previous match (consecutive run)
- +8 if it starts a word (`Go **t**o **S**ettings`)
- small capped penalty for gaps
- +10 if the whole query is a substring

Return the matched indices for highlighting. Search titles first, keywords as a weaker fallback. Sort by score, then shorter title.

```
export function fuzzyMatch(query, text) {
  const q = query.toLowerCase().replace(/\s+/g, '');
  const t = text.toLowerCase();
  const indices = [];
  let score = 0, from = 0;
  for (const ch of q) {
    const i = t.indexOf(ch, from);
    if (i === -1) return null;
    score += 1;
    if (i === indices.at(-1) + 1) score += 5;
    if (i === 0 || WORD_BREAK.test(t[i - 1])) score += 8;
    score -= Math.min(i - from, 3);
    indices.push(i);
    from = i + 1;
```

```
  }
  if (t.includes(query.toLowerCase().trim())) score += 10;
  return { score, indices };
}
```

Highlight without `innerHTML` (no XSS):

```
export function Highlight({ text, indices }) {
  if (!indices?.length) return text;
  const matched = new Set(indices);
  return text.split('').map((ch, i) => (matched.has(i) ? <mark key={i}>{ch}</mark> : ch));
}
```

Challenge 2: Focus management (trap, restore, IME)

Problem: While the modal is open, Tab must not walk into the page behind it, screen readers must not read the page, and on close the user must land back where they were (e.g. on the button that opened it). Building a focus trap by hand means tracking the first/last focusable elements and `aria-hidden` on siblings.

Solution: Use the platform. `dialog.showModal()` puts the dialog in the top layer and makes the rest of the page **inert** (not focusable, not clickable, hidden from assistive tech), and fires `cancel` on Escape. We remember `document.activeElement` before opening and focus it again on unmount. Inside, the input is the only tab stop, so Tab is prevented.

```
useEffect(() => {
  const dialog = dialogRef.current;
  const opener = document.activeElement;
  dialog.showModal();
  inputRef.current.focus();
  return () => {
    dialog.close();
    opener?.focus?.();
  };
}, []);

<dialog
  onCancel={(e) => { e.preventDefault(); onClose(); }}     // Esc → React owns closing
  onClick={(e) => e.target === e.currentTarget && onClose()} // backdrop
>
```

And in the key handler: `if (e.nativeEvent.isComposing) return;` so pressing Enter to confirm an IME composition does not run a command.

Challenge 3: Keyboard navigation that stays in sync with the mouse and the scroll

Problem 1: Focus must stay in the input (so typing keeps working), yet the screen reader must announce the active option. **Problem 2:** With a long list, Arrow Down moves the active row out of view. **Problem 3:** When the list scrolls under a **still** mouse cursor, `mouseenter` fires on the row that slides under it and steals the selection from the keyboard.

Solution:

- `aria-activedescendant` on the input points to the active `<li role="option">`; DOM focus never leaves the input.
- After `activeIndex` changes, `scrollIntoView({ block: 'nearest' })` on the selected row (only scrolls if needed).
- Use `onMouseMove`, not `onMouseEnter`: it only fires when the mouse really moves. `onMouseDown` prevents default so a click does not steal focus from the input.
- Wrap with modulo: `(i + step + items.length) % items.length`.

```
function onKeyDown(e) {
  if (e.nativeEvent.isComposing) return;
  if (e.key === 'ArrowDown' || e.key === 'ArrowUp') {
    e.preventDefault(); // caret would jump to start/end
    if (!items.length) return;
    const step = e.key === 'ArrowDown' ? 1 : -1;
    setActiveIndex((i) => (i + step + items.length) % items.length);
  } else if (e.key === 'Enter') {
    e.preventDefault();
    if (items[activeIndex]) run(items[activeIndex]);
  } else if (e.key === 'Tab') {
    e.preventDefault();
  }
}

useEffect(() => {
  listRef.current?.querySelector('[aria-selected="true"]')?.scrollIntoView({ block: 'nearest' });
}, [activeIndex, items]);
```

Challenge 4: The global shortcut

Problem: Ctrl+K in Chrome focuses the address bar; the listener must work from anywhere (even inside other inputs) and be cleaned up; Mac uses Cmd.

Solution: One `keydown` listener on `window` in a small hook; check `metaKey || ctrlKey`, compare `e.key.toLowerCase()` (Shift/Caps Lock give `K`), `preventDefault()`, and toggle. The handler is wrapped in `useCallback` so the effect does not re-subscribe on every render.

```
export function useHotkey(key, handler) {
  useEffect(() => {
    function onKeyDown(e) {
      if ((e.metaKey || e.ctrlKey) && e.key.toLowerCase() === key) {
        e.preventDefault();
        handler();
      }
    }
    window.addEventListener('keydown', onKeyDown);
    return () => window.removeEventListener('keydown', onKeyDown);
  }, [key, handler]);
}
```

---

8. Implement in the correct order

#	Step	Working result after the step
1	Command data with `createCommands(actions)` and `App` with page/theme/toast	Demo app renders
2	`open` state + trigger button + `{open && <CommandPalette/>}`	Palette shows and hides
3	Native `<dialog>` + `showModal()` + Esc (`onCancel`) + backdrop click	Real modal, page inert
4	Input + full list grouped by section	Static palette
5	`useHotkey('k')` toggle	Cmd/Ctrl+K works
6	`fuzzyMatch` + `searchCommands` (test with asserts first)	Typing filters and ranks
7	`Highlight` component	Matched characters marked
8	`activeIndex` , arrows with wrap, Enter, `run()`	Fully keyboard usable
9	Hover with `onMouseMove` , click, `onMouseDown` preventDefault	Mouse works without stealing focus
10	ARIA (combobox, listbox, option, `aria-activedescendant`), scroll into view, focus restore, IME guard	Accessible
11	Recents via `localStorage` with try/catch	Recent group on top
12	Empty state, result count, dark theme styles, reduced motion	Polished

---

9. Cover all states and edge cases

State / edge case	Handling
Initial (closed)	Only the page and the trigger button with the shortcut hint
Open, empty query, no recents	All commands grouped by section, first active
Open, empty query, with recents	"Recent" group on top (a command can appear there and in its section; keys use `group-id`)
Searching (results)	Ranked flat "Results" group, section shown as a tag on the right
Empty results	"No commands match “…”"; arrows and Enter do nothing
Whitespace-only query	Treated as empty (`query.trim()`)
Query with spaces (`go dash`)	Spaces ignored for matching
Upper-case query	Case-insensitive
Match only in keywords	Shown, ranked lower, no highlight
Arrow past the ends	Wraps around
Long list	Scrollable list, active row kept in view
Mouse resting over list while arrowing	No selection jump (`mousemove`, not `mouseenter`)
Tab / Shift+Tab	Prevented; focus stays in the input
IME composition + Enter	Ignored
Escape / backdrop / Cmd+K again	Close without running anything
Close by any route	Focus returns to the opener
`localStorage` blocked or corrupt	try/catch, `Array.isArray` guard → empty recents
Recent id no longer exists	Filtered out (`.filter(Boolean)`)
Clipboard denied	`.catch` shows "Could not copy link"
Command text contains `<script>`	Rendered as text by React, never as HTML
React StrictMode double mount	Effect cleanup closes the dialog, so the second `showModal()` is valid

10. Final structure

```
_75_command_palette/
├── index.html
```

```
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── App.jsx                    # open state, hotkey, demo app, toast
    ├── index.css                  # theme tokens, palette styles
    ├── data/
    │   └── commands.js            # createCommands(actions)
    ├── hooks/
    │   └── useHotkey.js           # global Cmd/Ctrl + key
    ├── utils/
    │   ├── fuzzy.js               # fuzzyMatch, searchCommands (pure)
    │   └── recent.js              # loadRecent, saveRecent
    └── components/
        ├── CommandPalette.jsx     # dialog, search, keyboard nav, a11y
        └── Highlight.jsx          # <mark> matched characters
```

---

11. Full working code

Start from `npm create vite@latest _75_command_palette -- --template react` and replace `src/` with the files below.

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/App.jsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { CommandPalette } from './components/CommandPalette';
import { createCommands } from './data/commands';
import { useHotkey } from './hooks/useHotkey';

const IS_MAC = navigator.userAgent.includes('Mac');
```

```
export default function App() {
  const [open, setOpen] = useState(false);
  const [page, setPage] = useState('Home');
  const [theme, setTheme] = useState('light');
  const [toast, setToast] = useState('');

  useHotkey('k', useCallback(() => setOpen((o) => !o), []));

  useEffect(() => {
    document.documentElement.dataset.theme = theme;
  }, [theme]);

  useEffect(() => {
    if (!toast) return;
    const id = setTimeout(() => setToast(''), 2500);
    return () => clearTimeout(id);
  }, [toast]);

  const commands = useMemo(
    () =>
      createCommands({
        navigate: setPage,
        toggleTheme: () => setTheme((t) => (t === 'light' ? 'dark' : 'light')),
        notify: setToast,
      }),
    []
  );

  return (
    <div className="app">
      <header className="app__header">
        <strong>Acme</strong>
        <button className="app__trigger" onClick={() => setOpen(true)}>
          Search commands… <kbd>{IS_MAC ? '⌘' : 'Ctrl'} K</kbd>
        </button>
      </header>

      <main className="app__main">
        <h1>{page}</h1>
        <p>
          Press <kbd>{IS_MAC ? '⌘' : 'Ctrl'} K</kbd> anywhere to open the command palette.
        </p>
      </main>

      {open && <CommandPalette commands={commands} onClose={() => setOpen(false)} />}
```

```
      <div className="toast" role="status">
        {toast}
      </div>
    </div>
  );
}
```

src/data/commands.js

```
// Commands are plain data. `perform` closes over app actions injected here,
// so the palette never needs to know what a command actually does.
export function createCommands({ navigate, toggleTheme, notify }) {
  return [
    { id: 'nav-home', title: 'Go to Home', section: 'Navigation', keywords: ['start', 'landing'],
perform: () => navigate('Home') },
    { id: 'nav-dashboard', title: 'Go to Dashboard', section: 'Navigation', keywords: ['stats',
'overview'], perform: () => navigate('Dashboard') },
    { id: 'nav-projects', title: 'Go to Projects', section: 'Navigation', keywords: ['repos', 'work'],
perform: () => navigate('Projects') },
    { id: 'nav-settings', title: 'Go to Settings', section: 'Navigation', keywords: ['preferences',
'config'], perform: () => navigate('Settings') },
    { id: 'nav-profile', title: 'Go to Profile', section: 'Navigation', keywords: ['account', 'me'],
perform: () => navigate('Profile') },
    { id: 'act-new-project', title: 'Create new project', section: 'Actions', keywords: ['add',
'repo'], perform: () => notify('Project created') },
    { id: 'act-new-issue', title: 'Create issue', section: 'Actions', keywords: ['bug', 'ticket',
'task'], perform: () => notify('Issue created') },
    { id: 'act-invite', title: 'Invite teammate', section: 'Actions', keywords: ['member', 'share',
'user'], perform: () => notify('Invite sent') },
    {
      id: 'act-copy-url',
      title: 'Copy current URL',
      section: 'Actions',
      keywords: ['link', 'clipboard'],
      perform: () =>
        navigator.clipboard
          .writeText(location.href)
          .then(() => notify('Link copied'))
          .catch(() => notify('Could not copy link')),
    },
    { id: 'pref-theme', title: 'Toggle dark mode', section: 'Preferences', keywords: ['theme',
'light', 'appearance'], perform: toggleTheme },
    { id: 'pref-notifications', title: 'Mute notifications', section: 'Preferences', keywords:
['silent', 'alerts'], perform: () => notify('Notifications muted') },
```

```
    { id: 'help-shortcuts', title: 'Show keyboard shortcuts', section: 'Help', keywords: ['keys', 'hotkeys'], perform: () => notify('Ctrl/Cmd + K opens this palette') },
    { id: 'help-docs', title: 'Open documentation', section: 'Help', keywords: ['guide', 'manual'], perform: () => notify('Opening docs…') },
    { id: 'help-support', title: 'Contact support', section: 'Help', keywords: ['help', 'email', 'chat'], perform: () => notify('Support ticket opened') },
    { id: 'acc-logout', title: 'Log out', section: 'Account', keywords: ['sign out', 'exit'], perform: () => notify('Logged out') },
  ];
}
```

src/hooks/useHotkey.js

```
import { useEffect } from 'react';

// Cmd+key on Mac, Ctrl+key elsewhere. `handler` should be stable (useCallback).
export function useHotkey(key, handler) {
  useEffect(() => {
    function onKeyDown(e) {
      if ((e.metaKey || e.ctrlKey) && e.key.toLowerCase() === key) {
        e.preventDefault(); // stop the browser's own Ctrl+K (focus address bar)
        handler();
      }
    }
    window.addEventListener('keydown', onKeyDown);
    return () => window.removeEventListener('keydown', onKeyDown);
  }, [key, handler]);
}
```

src/utils/fuzzy.js

```
const WORD_BREAK = /[\s\-_/.]/;

// Subsequence match: every query char must appear in order in the text.
// Returns { score, indices } or null. Higher score = better match.
// shortcut: greedy left-most match can miss the best alignment; use a DP scorer (like fzf) if ranking complaints show up.
export function fuzzyMatch(query, text) {
  const q = query.toLowerCase().replace(/\s+/g, '');
  const t = text.toLowerCase();
  if (!q) return { score: 0, indices: [] };

  const indices = [];
  let score = 0;
```

```
  let from = 0;
  for (const ch of q) {
    const i = t.indexOf(ch, from);
    if (i === -1) return null;
    score += 1;
    if (i === indices.at(-1) + 1) score += 5; // consecutive run
    if (i === 0 || WORD_BREAK.test(t[i - 1])) score += 8; // start of a word
    score -= Math.min(i - from, 3); // small capped penalty for gaps
    indices.push(i);
    from = i + 1;
  }
  if (t.includes(query.toLowerCase().trim())) score += 10; // exact substring wins
  return { score, indices };
}

// Ranks commands by title match; keywords are a weaker fallback (no highlight).
export function searchCommands(commands, query) {
  const results = [];
  for (const command of commands) {
    const titleMatch = fuzzyMatch(query, command.title);
    if (titleMatch) {
      results.push({ command, score: titleMatch.score, indices: titleMatch.indices });
      continue;
    }
    const keywordMatch = (command.keywords ?? []).map((k) => fuzzyMatch(query, k)).filter(Boolean)[0];
    if (keywordMatch) results.push({ command, score: keywordMatch.score - 5, indices: [] });
  }
  // Array.sort is stable, so equal scores keep the original command order.
  return results.sort((a, b) => b.score - a.score || a.command.title.length - b.command.title.length);
}
```

src/utils/recent.js

```
const KEY = 'command-palette:recent';
const MAX = 5;

export function loadRecent() {
  try {
    const ids = JSON.parse(localStorage.getItem(KEY));
    return Array.isArray(ids) ? ids : [];
  } catch {
    return [];
  }
}
```

```
export function saveRecent(id) {
  const next = [id, ...loadRecent().filter((x) => x !== id)].slice(0, MAX);
  try {
    localStorage.setItem(KEY, JSON.stringify(next));
  } catch {
    // storage blocked (private mode / quota): recents are a nice-to-have
  }
  return next;
}
```

src/components/Highlight.jsx

```
// Wraps matched characters in <mark>. Text stays text (no innerHTML), so no XSS.
export function Highlight({ text, indices }) {
  if (!indices?.length) return text;
  const matched = new Set(indices);
  return text.split('').map((ch, i) => (matched.has(i) ? <mark key={i}>{ch}</mark> : ch));
}
```

src/components/CommandPalette.jsx

```
import { Fragment, useEffect, useId, useMemo, useRef, useState } from 'react';
import { searchCommands } from '../utils/fuzzy';
import { loadRecent, saveRecent } from '../utils/recent';
import { Highlight } from './Highlight';

// Mounted only while open, so query / activeIndex reset for free on every open.
export function CommandPalette({ commands, onClose }) {
  const dialogRef = useRef(null);
  const inputRef = useRef(null);
  const listRef = useRef(null);
  const listId = useId();
  const [query, setQuery] = useState('');
  const [activeIndex, setActiveIndex] = useState(0);
  const [recentIds] = useState(loadRecent);

  // Flat list of rows; `group` drives the section headers.
  const items = useMemo(() => {
    if (!query.trim()) {
      const recent = recentIds
        .map((id) => commands.find((c) => c.id === id))
        .filter(Boolean)
        .map((command) => ({ command, group: 'Recent', indices: [] }));
      return [...recent, ...commands.map((command) => ({ command, group: command.section, indices: []
```

```
}))];
    }
    return searchCommands(commands, query).map((r) => ({ ...r, group: 'Results' }));
  }, [commands, query, recentIds]);

  // Native modal <dialog>: top layer, inert background (the focus trap) and Esc -> cancel.
  useEffect(() => {
    const dialog = dialogRef.current;
    const opener = document.activeElement;
    dialog.showModal();
    inputRef.current.focus();
    return () => {
      dialog.close();
      opener?.focus?.(); // give focus back to whatever opened the palette
    };
  }, []);

  // Keep the active row visible while arrowing through a long list.
  useEffect(() => {
    listRef.current?.querySelector('[aria-selected="true"]')?.scrollIntoView({ block: 'nearest' });
  }, [activeIndex, items]);

  function run(item) {
    saveRecent(item.command.id);
    onClose();
    item.command.perform();
  }

  function onKeyDown(e) {
    if (e.nativeEvent.isComposing) return; // IME: Enter confirms the composition, not a command
    if (e.key === 'ArrowDown' || e.key === 'ArrowUp') {
      e.preventDefault(); // stop the caret jumping to start/end of the input
      if (!items.length) return;
      const step = e.key === 'ArrowDown' ? 1 : -1;
      setActiveIndex((i) => (i + step + items.length) % items.length);
    } else if (e.key === 'Enter') {
      e.preventDefault();
      if (items[activeIndex]) run(items[activeIndex]);
    } else if (e.key === 'Tab') {
      e.preventDefault(); // the input is the only tab stop; options use aria-activedescendant
    }
  }

  const activeId = items[activeIndex] ? `${listId}-${activeIndex}` : undefined;

  return (
```

```
    <dialog
      ref={dialogRef}
      className="palette"
      aria-label="Command palette"
      onCancel={(e) => {
        e.preventDefault(); // let React unmount us instead of the browser closing the dialog
        onClose();
      }}
      onClick={(e) => e.target === e.currentTarget && onClose()} // backdrop click
    >
      <div className="palette__panel" onKeyDown={onKeyDown}>
        <input
          ref={inputRef}
          className="palette__input"
          role="combobox"
          aria-expanded="true"
          aria-controls={listId}
          aria-activedescendant={activeId}
          aria-autocomplete="list"
          placeholder="Type a command or search…"
          value={query}
          onChange={(e) => {
            setQuery(e.target.value);
            setActiveIndex(0); // new results -> best match is active
          }}
        />

        {items.length ? (
          <ul id={listId} ref={listRef} role="listbox" className="palette__list">
            {items.map((item, i) => (
              <Fragment key={`${item.group}-${item.command.id}`}>
                {item.group !== items[i - 1]?.group && (
                  <li role="presentation" className="palette__group">
                    {item.group}
                  </li>
                )}
                <li
                  id={`${listId}-${i}`}
                  role="option"
                  aria-selected={i === activeIndex}
                  className="palette__item"
                  onMouseMove={() => setActiveIndex(i)} // not mouseenter: rows scrolling under a
still cursor must not steal the selection
                  onMouseDown={(e) => e.preventDefault()} // keep focus in the input
                  onClick={() => run(item)}
                >
```

```
                  <span>
                    <Highlight text={item.command.title} indices={item.indices} />
                  </span>
                  {item.group !== item.command.section && <span className="palette__section">
{item.command.section}</span>}
                </li>
              </Fragment>
            ))}
          </ul>
        ) : (
          <p className="palette__empty">No commands match “{query}”</p>
        )}

        <div className="palette__footer">
          <span aria-live="polite">{items.length} results</span>
          <span>
            <kbd>↑</kbd>
            <kbd>↓</kbd> navigate <kbd>↵</kbd> run <kbd>esc</kbd> close
          </span>
        </div>
      </div>
    </dialog>
  );
}
```

src/index.css

```
:root {
  --bg: #f6f7f9;
  --surface: #ffffff;
  --text: #1c1e21;
  --muted: #6b7280;
  --border: #e3e5e8;
  --accent: #4f46e5;
  --active: #eef0ff;
  --mark: #c7d2fe;
  font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
  color: var(--text);
  background: var(--bg);
}

:root[data-theme='dark'] {
  --bg: #111318;
  --surface: #1b1e25;
  --text: #e8eaed;
```

```
  --muted: #9aa0a6;
  --border: #2c3039;
  --accent: #818cf8;
  --active: #262b3a;
  --mark: #3730a3;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
}

kbd {
  font: 12px ui-monospace, monospace;
  padding: 1px 6px;
  border: 1px solid var(--border);
  border-radius: 4px;
  background: var(--bg);
  color: var(--muted);
}

.app__header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 24px;
  background: var(--surface);
  border-bottom: 1px solid var(--border);
}

.app__trigger {
  display: flex;
  gap: 24px;
  align-items: center;
  padding: 6px 10px;
  font: inherit;
  color: var(--muted);
  background: var(--bg);
  border: 1px solid var(--border);
  border-radius: 8px;
  cursor: pointer;
}
```

```
.app__main {
  padding: 32px 24px;
}

/* ---------- palette ---------- */
.palette {
  width: min(640px, calc(100vw - 32px));
  margin: 12vh auto auto;
  padding: 0; /* panel fills the dialog, so a click on the dialog itself = backdrop */
  border: 1px solid var(--border);
  border-radius: 12px;
  background: var(--surface);
  color: var(--text);
  box-shadow: 0 20px 60px rgb(0 0 0 / 0.25);
}

.palette::backdrop {
  background: rgb(0 0 0 / 0.4);
}

.palette__input {
  width: 100%;
  padding: 16px 20px;
  font: inherit;
  font-size: 17px;
  color: inherit;
  background: transparent;
  border: 0;
  border-bottom: 1px solid var(--border);
  outline: none;
}

.palette__list {
  max-height: 360px;
  overflow-y: auto;
  margin: 0;
  padding: 6px;
  list-style: none;
}

.palette__group {
  padding: 10px 12px 4px;
  font-size: 12px;
  font-weight: 600;
  color: var(--muted);
  text-transform: uppercase;
```

```
  letter-spacing: 0.04em;
}

.palette__item {
  display: flex;
  justify-content: space-between;
  padding: 10px 12px;
  border-radius: 8px;
  cursor: pointer;
}

.palette__item[aria-selected='true'] {
  background: var(--active);
  box-shadow: inset 3px 0 0 var(--accent);
}

.palette__item mark {
  background: var(--mark);
  color: inherit;
  border-radius: 2px;
}

.palette__section {
  font-size: 12px;
  color: var(--muted);
}

.palette__empty {
  margin: 0;
  padding: 32px 20px;
  text-align: center;
  color: var(--muted);
}

.palette__footer {
  display: flex;
  justify-content: space-between;
  gap: 8px;
  flex-wrap: wrap;
  padding: 8px 16px;
  font-size: 12px;
  color: var(--muted);
  border-top: 1px solid var(--border);
}

.toast:empty {
```

```
  display: none;
}

.toast {
  position: fixed;
  bottom: 24px;
  left: 50%;
  transform: translateX(-50%);
  padding: 10px 16px;
  border-radius: 8px;
  background: var(--text);
  color: var(--bg);
}

@media (prefers-reduced-motion: no-preference) {
  .palette[open] {
    animation: pop 120ms ease-out;
  }
}

@keyframes pop {
  from {
    opacity: 0;
    transform: scale(0.98);
  }
}
```

Quick self-check of the search logic (node)

```
// check.mjs, run with: node check.mjs
import assert from 'node:assert/strict';
import { fuzzyMatch, searchCommands } from './src/utils/fuzzy.js';
import { createCommands } from './src/data/commands.js';

assert.deepEqual(fuzzyMatch('gts', 'Go to Settings').indices, [0, 3, 6]);
assert.equal(fuzzyMatch('xyz', 'Go to Settings'), null);
assert.equal(fuzzyMatch('so', 'Go to Settings'), null); // order matters
assert.ok(fuzzyMatch('set', 'Go to Settings').score > fuzzyMatch('set', 'Show keyboard
shortcuts').score); // word start + substring

const cmds = createCommands({ navigate() {}, toggleTheme() {}, notify() {} });
assert.equal(searchCommands(cmds, 'settings')[0].command.id, 'nav-settings');
assert.equal(searchCommands(cmds, 'dark')[0].command.id, 'pref-theme');
assert.equal(searchCommands(cmds, 'theme')[0].command.id, 'pref-theme'); // keyword fallback
assert.deepEqual(searchCommands(cmds, 'theme')[0].indices, []);
```

```
assert.equal(searchCommands(cmds, 'go dash')[0].command.id, 'nav-dashboard'); // spaces ignored
assert.equal(searchCommands(cmds, 'qqqq').length, 0);
assert.equal(searchCommands(cmds, 'GO TO PRO')[0].command.id, 'nav-profile'); // case-insensitive
console.log('top for "gp":', searchCommands(cmds, 'gp').slice(0, 3).map((r) => r.command.title));
console.log('top for "cr":', searchCommands(cmds, 'cr').slice(0, 3).map((r) => r.command.title));
console.log('all fuzzy checks passed');
```

---

12. Wrap-up

Edge cases handled

Wrap-around navigation, empty results, whitespace and case in queries, keyword-only matches, IME Enter, Tab trap, focus restore, blocked storage, stale recent ids, clipboard failure, XSS-safe highlight, StrictMode double mount, mouse vs keyboard selection conflict.

Trade-offs to mention

- **Native `<dialog>` vs a custom portal + manual focus trap:** native gives inert background, top layer, Esc and backdrop for free with less code; a manual trap gives full control of animation and older-browser support.
- **Greedy fuzzy match vs optimal alignment (DP, like fzf):** greedy is O(n) per command and simple; it can miss the best-scoring alignment for some queries.
- **Mount-on-open vs always mounted:** free state reset and no hidden DOM, but no exit animation and recents are re-read each open (cheap).
- **`aria-activedescendant` vs moving real focus to options:** typing keeps working and screen readers still announce the option; needs correct ids.
- **Recents in `localStorage`:** per device only; a server would sync across devices.
- **No debounce:** filtering a few hundred items in memory is faster than a frame; debounce only adds lag.

What to add with more time

- **Tests:** unit tests for `fuzzyMatch` / `searchCommands`; React Testing Library for open, type, arrows, Enter, Escape, focus restore; axe for a11y.
- **Nested pages:** a stack of pages (`Change theme → Light / Dark`) with Backspace on empty input to go back.
- **Async sources:** search files/users from an API with `AbortController` , loading row, and `useDeferredValue` .
- **Large lists:** virtualize the list (only render visible rows) for thousands of commands.
- **Per-command shortcuts** shown as `<kbd>` and registered globally; context-aware commands (only show "Delete issue" on an issue page).
- **Exit animation**, and a command registry (context/provider) so any feature can register commands.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why fuzzy search and not a simple `includes`?

Users type abbreviations: `gts` for *Go to Settings*, `tdm` for *Toggle dark mode*. A command palette is about speed, so subsequence matching with word-start bonuses is the expected behavior (VS Code, Linear, Notion all do it). `includes` is the fallback I'd ship first if time is short.

Q2. What if the commands depend on where the user is (e.g. "Delete issue" only on an issue page)?

Give each command an optional `when(context)` predicate and filter by it before searching, or let each page register its commands into a registry context on mount and remove them on unmount. The palette itself does not change; it still gets `commands`.

Q3. How would you support nested commands like *Change theme → Light / Dark*?

Keep a stack of pages in the palette: `const [pages, setPages] = useState([root])`. A command can have `children` instead of `perform`; running it pushes a page and clears the query. Backspace on an empty query pops. The breadcrumb shows the stack.

State design

Q4. Why not store the filtered results in state?

They are a pure function of `commands` and `query`. Storing them means an extra render per keystroke and a chance for the list and the query to disagree. `useMemo` derives them in the same render.

Q5. Why store `activeIndex` and not the active command id?

The list is re-built on every keystroke and we always want the top result active then, so an index that resets to 0 is simplest. If results could update while the user is navigating (async results), I would store the id instead so the selection sticks to the same command when rows above it move.

Q6. Why reset `activeIndex` in `onChange` and not in a `useEffect` on `query`?

An effect runs after the render, so for one frame the old index points into the new list (it could be out of range or on the wrong row) and it causes a second render. Setting both in the event handler batches them into one consistent render.

Q7. Why is the palette unmounted when closed instead of being hidden?

Unmounting resets `query` and `activeIndex` for free, removes all listeners, and keeps the DOM clean. The cost is no exit animation; for that I'd keep it mounted and reset state in the close handler, or use a `data-`

`state="closed"` animation and unmount on `animationend`.

Fuzzy search challenge

Q8. What if there are 1 million items (e.g. searching all files)?

Don't run fuzzy over everything on the main thread per keystroke. Options: pre-filter with an index (trigrams / first-letter buckets), run matching in a **Web Worker**, use `useDeferredValue` so typing stays responsive, cap results to the top 50 with a partial sort, and **virtualize** the list. Often the real answer is: search on the server.

Q9. What is the complexity of your matcher, and its weakness?

O(len(text)) per command because `indexOf` only moves forward, so O(total characters) per keystroke. The weakness is it is greedy: it takes the left-most match for each character, which can miss a better alignment (e.g. matching a later word start). A DP scorer like fzf's finds the optimal alignment at O(q × t) cost.

Q10. How would you handle typos (`settigns`)?

Subsequence matching can't. Add an edit-distance (Levenshtein / Damerau) fallback for short queries when there are no fuzzy hits, or use a library like Fuse.js. Typo tolerance adds noise, so rank those results below real matches.

Q11. Why not set the highlight with `dangerouslySetInnerHTML` and a regex replace?

Command titles could come from user content (project names). Building HTML strings opens XSS. Mapping characters to `<mark>` elements keeps everything as text that React escapes.

Focus and accessibility challenge

Q12. Why the native `<dialog>` instead of a div with a portal?

`showModal()` gives the top layer (no z-index fights), makes the rest of the page **inert** (the real focus trap, including screen readers), Escape via `cancel`, and a `::backdrop`. A div needs a portal, a manual Tab trap, `aria-modal`, `aria-hidden` on siblings and a scroll lock.

Q13. How would you write the focus trap by hand if `<dialog>` wasn't allowed?

On keydown Tab inside the modal: find focusable elements (`button, [href], input, [tabindex]:not([tabindex="-1"])`), if Shift+Tab on the first, focus the last; if Tab on the last, focus the first. Plus set `inert` (or `aria-hidden`) on the app root while open and restore focus on close.

Q14. Why `aria-activedescendant` instead of moving focus to each option?

If focus moved to the option, typing would stop going into the input. With `aria-activedescendant` the input keeps focus and the screen reader announces the option whose id it points to. The roles must be right: input `combobox`, list `listbox`, rows `option` with `aria-selected`.

Q15. Why `onMouseMove` and not `onMouseEnter` on the rows?

When you arrow down, the list scrolls, and a different row slides under a cursor that did not move. `mouseenter` fires for it and the selection jumps back to the mouse. `mousemove` only fires on real movement.

Q16. What is the `isComposing` check for?

With Japanese/Chinese/Korean input methods, Enter confirms the composed characters. Without the check, that Enter would also run the active command. `e.nativeEvent.isComposing` is true during composition.

Implementation steps

Q17. What happens in React StrictMode, where effects run twice?

Mount → `showModal()`, StrictMode cleanup → `dialog.close()` + focus restore, mount again → `showModal()`. Because cleanup closes the dialog, the second `showModal()` is valid. Without the close in cleanup, it would throw "already open" in some browsers.

Q18. Why does `useHotkey` require a stable handler?

The handler is in the effect's dependency list. A new function every render would remove and re-add the window listener on every render. `useCallback(() => setOpen(o => !o), [])` keeps it stable; the functional update means it never needs `open` in its closure. (A `useRef`-held latest handler, a "useEvent" pattern, removes the requirement.)

Q19. Why close the palette before calling `perform()`?

The command may navigate, open another modal, or move focus. If the palette is still mounted, its focus-restore would fight that. Closing first, then running, keeps the order predictable. For commands that should keep the palette open (multi-select), add a `keepOpen` flag.

Q20. What if `perform` is async and can fail (e.g. "Create issue" calls an API)?

Close the palette and show a pending toast, then success or error. Or keep the palette open with a spinner on the row, disable Enter while pending, and show the error inline. Never swallow the rejection silently.

Performance

Q21. Do you need debounce on the search input?

Not for in-memory data: filtering a few hundred items takes well under a millisecond, and debounce would only make the list lag behind the typing. Debounce (with `AbortController`) is for network searches.

Q22. What re-renders when the active row changes?

The palette re-renders, and with it every row. For a few hundred rows that is fine. For thousands: memoize the row component (`React.memo`) so only the two rows whose `selected` prop changed re-render, and virtualize.

Trade-offs and twists

Q23. What if the palette must work on mobile?

No Cmd+K there, so the visible trigger button matters. Make the dialog full-width near the top, make rows taller for touch, and keep the keyboard open by focusing the input. Hover is irrelevant; tap = run.

Q24. What if two components both register Cmd+K, or a text editor inside the page uses it?

Have one owner of global shortcuts (a shortcut registry) that resolves conflicts, or check `e.defaultPrevented` / the focused element before acting, so an editor that handles Cmd+K itself can call `preventDefault()` and the palette skips it.

Q25. How would you test this?

Unit-test `fuzzyMatch` and `searchCommands` with plain asserts (ranking, null on no match, keyword fallback). With React Testing Library: press Ctrl+K → dialog open and input focused; type → options filtered; ArrowDown/Enter → command mock called and dialog gone; Escape → closed and focus back on the trigger. Run axe on the open dialog.